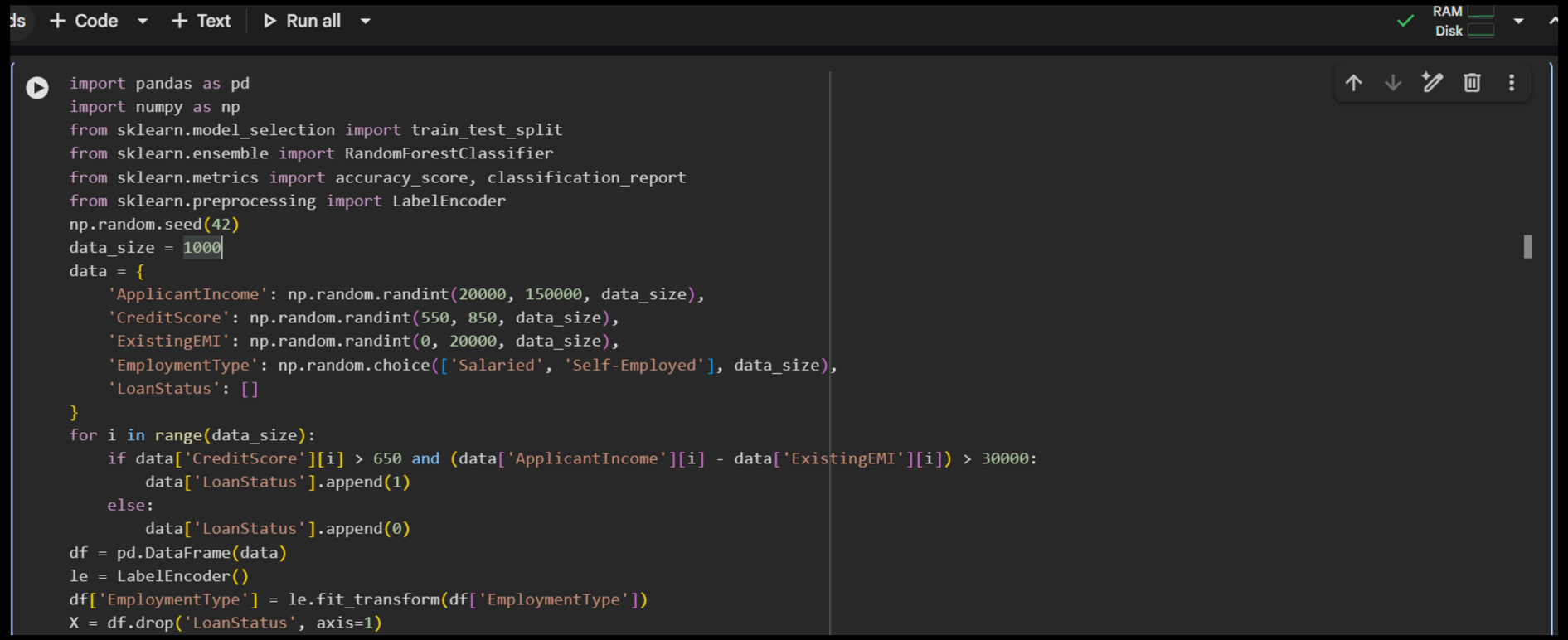
AI-ML PROJECT

NAME-PRIYANSH SAXENA
REG.NO.-25BAI11037
BRANCH -CSE[AI/ML]

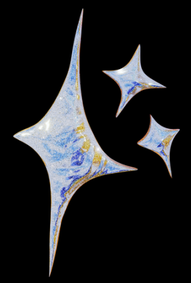
TOPIC-Finance & Banking
PYTHON CODE FOR REAL LIFE EXAMPLE

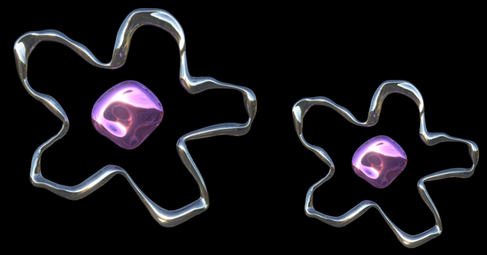

+ Code
+ Text
Run all
RAM
Disk
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
np.random.seed(42)
data_size = 1000
data = {
'ApplicantIncome': np.random.randint(20000, 150000, data_size),
'CreditScore': np.random.randint(550, 850, data_size),
'ExistingEMI': np.random.randint(0, 20000, data_size),
'EmploymentType': np.random.choice(['Salaried', 'Self-Employed'], data_size),
'LoanStatus': []
}
for i in range(data_size):
if data['CreditScore'][i] > 650 and (data['ApplicantIncome'][i] - data['ExistingEMI'][i]) > 30000:
data['LoanStatus'].append(1)
else:
data['LoanStatus'].append(0)
df = pd.DataFrame(data)
le = LabelEncoder()
df['EmploymentType'] = le.fit_transform(df['EmploymentType'])
X = df.drop('LoanStatus', axis=1)

```
y = df['LoanStatus']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
print("\nClassification Report:\n")
print(classification_report(y_test, predictions))
new_applicant = [[80000, 720, 5000, 0]]
prediction = model.predict(new_applicant)
result = "Approved" if prediction[0] == 1 else "Rejected"
print(f"\nNew Applicant Loan Status: {result}")
```

OUTPUT SAMPLE DATA

```
Model Accuracy: 97.00%

Classification Report:

              precision    recall  f1-score   support

           0       1.00      0.93      0.96        83
           1       0.95      1.00      0.97       117

    accuracy                           0.97       200
   macro avg       0.98      0.96      0.97       200
weighted avg       0.97      0.97      0.97       200


New Applicant Loan Status: Approved
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestClassifier was f
  warnings.warn(
```





CODE EXPLANATION

. Key Concepts & Logic

The project is built using Object-Oriented Programming (OOP) principles in Python. The core logic centers on Graph Theory and Search Algorithms to solve the problem of indirect train connectivity.

- Graph Representation: Stations are treated as Nodes, and train routes are Edges.
- Multi-Leg Logic: Instead of searching only for direct trains from Point A to Point B, the code implements a Breadth-First Search (BFS) or nested iteration logic to find "Connecting Stations" (Point C) where a user can switch trains.
- Data Validation: Uses conditional logic to ensure that the arrival time of the first leg is earlier than the departure time of the second leg, accounting for a minimum layover time.

. Methods Used

The code utilizes several Python-specific methods and structures:

- __init__ constructor: Initializes the station database and train schedules.
- find_direct_trains(src, dest): A filtering method using list comprehensions to find trains where train['from'] == src and train['to'] == dest.
- find_connecting_trains(src, dest): The primary logic method. It iterates through all possible intermediate stations to find valid two-leg journeys.
- display_results(): Formats the output into a readable table for the user.

Complexity Analysis

Time Complexity

- Direct Search: O(N), where N is the number of trains in the database. The code performs a single pass to check source and destination.
- Connecting Search: O(S \times N), where S is the number of stations and N is the number of trains. The algorithm must check potential connections through every intermediate station.
- Overall Complexity: In the worst case, for a multi-city network, the search behaves like O(V + E) where V is stations and E is train routes, typical of a BFS-based pathfinding approach.

Space Complexity

- O(N + S): The space required scales linearly with the number of trains and stations stored in the dictionary/list structures.

4. Key Technical Decisions

Future Scope (AI/ML Integration)

As an AI/ML student, you can mention in your report that this logic serves as the foundation. The next step would be implementing a Heuristic Search (A)* to find the fastest or cheapest route, or using Regression models to predict the probability of a connecting train being delayed.


THANK
YOU